

TSS: Technical Report and Architecture Design

Triple-Stream Stress Probe

Transferable Stress Signals

A Comprehensive Engineering Reference
for the ACL BioNLP 2026 Submission:

*“The Divergence Hypothesis: Unmasking Lexical Interference
and Label Bias in Mental Health NLP”*

Version 1.0.0

System: TSS (Triple-Stream Stress Probe)

Codebase: 11,651 lines across 11 Python modules

Pipeline: 6-stage automated workflow (`run_all.sh`)

Datasets: $N = 12,906$ across 4 corpora, 2 labeling regimes

Channels: 3 independent feature streams (A, B, C) $\rightarrow$ 7 combinations

Contents

1	The Epistemic Framework: Why TSS Exists	3
1.1	The Generalization Crisis in Computational Mental Health	3
1.2	The Illusion of Distant Supervision	3
1.3	The Four Complementary Hypotheses	3
1.4	Positioning as an Audit Probe	4
2	Data Engineering and Decontamination	4
2.1	The Reality-Preserving Philosophy	4
2.2	Zero-Leakage Cross-Dataset Decontamination	5
2.3	Feature Engineering Pipeline	5
3	Channel Architecture and Feature Extraction	5
3.1	Channel A: Lexical Surface-Form (500 features)	5
3.2	Channel B: Content-Free Morpho-Syntax (6 features)	6
3.2.1	POS Bigram Extraction	6
3.2.2	Abstract POS-SVO Triples	6
3.2.3	Adaptive Length-Smoothing	7
3.2.4	The Six Features of Channel B	7
3.3	Channel C: Psycholinguistic Style (154 features)	8
3.3.1	Yule’s I: Length-Independent Lexical Diversity	8
3.3.2	Structural Negation Scope Balance	8
3.3.3	Sentence Rhythm Features	8
3.3.4	Length-Robust Squashing Transform	8
3.3.5	Open Lexicon Categories (112 features)	9
4	The ML Pipeline and Algorithmic Decisions	9
4.1	L2 Normalization: From Quantity to Density	9
4.2	Conditional Scaling: Protecting Channel B’s Sparsity	9
4.3	ElasticNet and the Structural Sparsity Bias	10
4.4	Dynamic Threshold Calibration	10
5	Audit Protocols and Statistical Inference	11
5.1	Degree of Divergence (DoD): The DiD Adaptation	11
5.2	The Lexical Interference Paradox: Statistical Evidence	11
5.3	Interventional Masking: Quasi-Causal Evidence	12
5.4	The Permutation Tension	13
5.5	Unsupervised Phenotyping and Leakage Auditing	13
5.6	Channel Orthogonality	14
5.7	LLM Baseline Instability: Regime-Sensitive Collapse	14
5.8	LODO Cross-Stressor Evaluation: Detailed Protocol	14
6	Qualitative Conflict Zone Analysis	15
6.1	Methodology: Systematic Shift-Case Extraction	15
6.2	Three Failure Archetypes	15
6.2.1	Archetype 1: Semantic Hallucination (Keyword Spotting)	15
6.2.2	Archetype 2: Implicit Stress (Missing Vocabulary)	15
6.2.3	Archetype 3: Boundary Collapse (Context Contagion)	16

7	Efficiency, Privacy, and Reproducibility	16
7.1	Green AI: Training Time Analysis	16
7.2	Privacy-by-Design	16
7.3	Reproducibility Checklist	17
7.4	Pipeline Execution	17
8	Dataset Registry and Auto-Discovery	17
9	Codebase Architecture Map	17
9.1	Module Dependency Graph	17
9.2	Data Flow	18
9.3	Key Design Patterns	18
10	Conclusion: Engineering Philosophy	18
10.1	Summary of Key Engineering Decisions	19

1 The Epistemic Framework: Why TSS Exists

1.1 The Generalization Crisis in Computational Mental Health

Computational Mental Health (CMH) models routinely report impressive benchmark performance—MentalBERT achieves 0.801 Macro-F1 on Dreaddit-test, and LLaMA-3 reaches 0.800 on Twitter-gold. Yet these same models collapse catastrophically under distribution shift: MentalBERT drops to 0.418 on Twitter-auto, and LLaMA-3’s recall plummets from 0.984 (Dreaddit) to 0.203 (Twitter-auto).

This is not a mere engineering problem. It is an **epistemic crisis**: the models are not learning “stress” in any clinically meaningful sense. They are learning *which words co-occur with the label “stressed”* in a particular corpus. When that co-occurrence pattern shifts—because the platform changes, or because the labeling method changes—the model fails.

Core Philosophical Position

TSS is **not** a SOTA classifier. It is an **epistemic audit filter**—a diagnostic instrument that measures *whether your model is measuring what you think it’s measuring*. It quantifies the gap between what auto-labels reward (lexical keywords) and what human annotators reward (structural/stylistic signals).

1.2 The Illusion of Distant Supervision

Distant supervision in CMH typically operates through proxy signals: a post from `r/anxiety` is labeled “stressed,” a tweet containing “#mentalhealth” is labeled “stressed.” This creates a systematic confound: the label is assigned based on *topical content* (what the text is about), but clinical stress is a *cognitive-affective state* (how the person is thinking and feeling).

The consequence is what we term **Labeling-Regime Divergence (LRD)**: human annotators and distant supervision reward fundamentally different linguistic channels. Human annotators attend to fragmentation, hedging, negation patterns, absolutist language—signals that are largely *unconscious* and therefore difficult to fake. Distant supervision, by contrast, rewards keyword presence—signals that are *conscious* and easily gamed.

1.3 The Four Complementary Hypotheses

TSS operationalizes LRD through four testable hypotheses:

- H1 Semantic-Structural Intersection (SSI).** In closed support communities, stress keywords co-occur with genuine structural distress. This coincidence enables shortcut learning. When SSI weakens (mixed-intent platforms like Twitter), lexical models degrade.
- H2 Cross-Stressor Cognitive Invariance.** If stress is a *state* rather than a *topic*, structural markers should transfer across stressor domains (abuse, anxiety, financial, PTSD, social). The LODO protocol tests this directly.
- H3 Implicit Stress vs. Semantic Hallucination.** Lexical systems confuse meta-talk about stress with actual distress (semantic hallucination), and miss “silent stress” where no explicit lexicon appears. Structural probes should reduce both failure modes.
- H4 Structural Sparsity as an Unconscious Mirror.** If structural traces are cognitively diagnostic, learning dynamics should compress toward sparse representations. The consistent convergence of Channel B to $l_1_ratio = 0.95$ (near-Lasso) confirms this algorithmically.

1.4 Positioning as an Audit Probe

The critical design consequence is that TSS *intentionally* does not compete on SOTA metrics. Its Macro-F1 of 0.739 (Channel C on Dreddit) is lower than MentalBERT’s 0.801. This is by design: TSS trades raw performance for **interpretability**, **diagnostic power**, and **regime-awareness**. The DoD metric it produces (0.0374, $p = 0.0032$) is a measurement that no black-box model can provide: a statistically grounded scalar quantifying *how much your labels are lying to you*.

2 Data Engineering and Decontamination

01_prepare_data.py — 1,306 lines

2.1 The Reality-Preserving Philosophy

Anti-Pattern: Default Undersampling

Many CMH pipelines routinely undersample the majority class to achieve 50/50 balance. TSS **rejects** this approach for headline results. Reddit-combi has $\sim 88\%$ non-stressed prevalence—this is the *epidemiological reality*. Artificially deleting 88% of non-stressed samples distorts the base rate and inflates apparent model performance.

Instead, TSS shifts the burden from data deletion to algorithmic correction:

- `class_weight='balanced'` in LogisticRegression inversely weights samples by class frequency
- **Dynamic threshold calibration** (Section 4.4) optimizes the decision boundary on out-of-fold probabilities
- **Macro-F1** as the primary metric treats both classes equally regardless of prevalence

The majority-class baselines confirm this works. Below is the formal derivation of why the majority-class baseline for Macro-F1 is not simply “1.0” even when predicting a single class:

Majority-Class Baseline: The Mathematical Floor

For a dataset with prevalence π (fraction of class 1), predicting all-majority yields:

- If majority is class 0 ($\pi < 0.5$): $\text{Precision}_1 = 0, \text{Recall}_1 = 0 \Rightarrow F1_1 = 0; F1_0 = \frac{2(1-\pi)}{2-\pi}$
- $\text{Macro-F1} = \frac{F1_0 + F1_1}{2} = \frac{1}{2} \cdot \frac{2(1-\pi)}{2-\pi}$

For Reddit-combi ($\pi = 0.120$): $\text{Macro-F1}_{\text{majority}} = \frac{1}{2} \cdot \frac{2 \times 0.88}{1.88} = 0.468$. This is the floor all TSS channels must exceed.

Table 1: Epidemiological reality: class distributions and majority baselines.

Dataset	Label Regime	Prevalence (π)	Majority Macro-F1	N
Dreddit-test	Human	0.516	0.340	715
Twitter-gold	Human	0.338	0.398	2,863
Twitter-auto	Auto	0.478	0.343	6,218
Reddit-combi	Auto	0.120	0.468	3,110
Total evaluation instances				12,906

2.2 Zero-Leakage Cross-Dataset Decontamination

Data leakage between training (Dreaddit-train) and evaluation sets (Dreaddit-test, Twitter-gold) is a pervasive threat in cross-platform CMH evaluation. TSS implements strict decontamination:

Listing 1: Cross-dataset decontamination (01_prepare_data.py)

```

1 def remove_cross_duplicates(train_df, test_df, text_col='cleaned_text'):
2     """Remove exact text matches between train and test."""
3     train_texts = set(train_df[text_col].values)
4     mask = ~test_df[text_col].isin(train_texts)
5     removed = (~mask).sum()
6     logger.info(f"Removed {removed} duplicates from test set")
7     return test_df[mask].reset_index(drop=True)

```

Additionally, `duplicates_removed=true` enforces within-dataset deduplication by exact text match. The pipeline logs every removal for auditability.

2.3 Feature Engineering Pipeline

The data preparation script extracts ~ 30 base psycholinguistic features per text, all strictly **length-normalized**:

- **Morphological:** POS ratios (pronoun/token, verb/token, adjective/token, adverb/token, modal/token, negation/token)
- **Stylistic:** Caps ratio, punctuation density, elongated word count
- **Affective:** NRC-VAD (Valence, Arousal), VADER compound, emotion densities (fear, anger, sadness, joy per token)
- **Complexity:** Flesch-Kincaid grade, Gunning Fog index, Type-Token Ratio
- **Cognitive:** Absolutist word ratio, first-person pronoun ratio, stress verb presence
- **Structural:** SVO triples per sentence, fragmentation index, average sentence length

Every ratio feature divides by `max(token_count, 1)` to prevent division by zero while preserving length-invariance.

3 Channel Architecture and Feature Extraction

`tss/features.py` — 1,184 lines

This is the engineering heart of TSS. Three channels decompose text into orthogonal linguistic planes.

3.1 Channel A: Lexical Surface-Form (500 features)

Design Decision: Why Character N-grams?

Channel A uses character-level TF-IDF (3–5 grams) rather than word-level features. Character n-grams capture sub-word patterns (misspellings, informal language, morphological variants) that word-level tokenization would miss. This makes Channel A more robust to social media text while still capturing lexical content.

Pipeline:

1. `TfidfVectorizer(analyzer='char_wb', ngram_range=(3,5), max_features=3000)`
2. `SelectKBest(chi2, k=500)` selects the 500 most discriminative n-grams

The `char_wb` analyzer respects word boundaries, preventing cross-word artifacts. Chi-square selection reduces the 3,000-dimensional space to 500 features, trading some lexical coverage for computational efficiency and reduced overfitting.

3.2 Channel B: Content-Free Morpho-Syntax (6 features)

Channel B is the most architecturally novel component of TSS. It operates exclusively on POS tags and abstract SVO triples—**no lexical tokens are ever stored or processed**.

3.2.1 POS Bigram Extraction

Listing 2: POS bigram extraction (privacy-preserving)

```

1 def _extract_pos_bigrams(self, pairs):
2     # Input: [('I', 'PRP', 'i'), ('feel', 'VBP', 'feel'), ('bad', 'JJ', 'bad')]
3     # Output: ['PRP_VBP', 'VBP_JJ']
4     # NOTE: actual words are NEVER used
5     bigrams = []
6     prev_pos = None
7     for item in pairs:
8         current_pos = item[1] # POS tag only
9         if prev_pos is not None:
10            bigrams.append(f"{prev_pos}_{current_pos}")
11            prev_pos = current_pos
12     return bigrams

```

Design Decision: Why POS Bigrams Instead of Unigrams?

The original system used token+POS unigrams (e.g., `feel_VB`), which leaked lexical content into Channel B. POS bigrams (e.g., `PRP_VBP`) capture purely structural patterns: “pronoun followed by verb” rather than “the word ‘feel’ as a verb.” This is critical for:

- **Privacy-by-design:** No patient text is stored or transmitted
- **Domain agnosticism:** The pattern `PRP_VBP` captures “I feel,” “I hate,” “I love” equally
- **Naturally small vocabulary:** $\sim 25 \text{ POS tags} \times 25 = 625$ max bigrams, eliminating the need for entropy filtering

3.2.2 Abstract POS-SVO Triples

Channel B also extracts Subject-Verb-Object triples via spaCy dependency parsing, but retains *only* the POS tags:

Listing 3: Content-free SVO extraction

```

1 # "I hate stress" -> "SVO_PRON_VERB_NOUN"
2 # "She cried"      -> "SV_PRON_VERB"
3 # NEVER: "SVO_I_hate_stress"
4 subj_pos = subject.pos_ # e.g., "PRON"
5 verb_pos = root.pos_    # e.g., "VERB"

```

```

6 obj_pos = obj.pos_          # e.g., "NOUN"
7 triple = f"SVO_{subj_pos}_{verb_pos}_{obj_pos}"

```

3.2.3 Adaptive Length-Smoothing

The most critical engineering decision in Channel B addresses cross-platform length bias. A Reddit post might contain 200 tokens with 199 POS bigrams; a tweet might contain 15 tokens with 14 bigrams. Without normalization, the model would systematically assign higher scores to longer texts.

$$\lambda = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) \cdot \min\left(\frac{L}{L_{\text{ref}}}, 1\right) \quad (1)$$

where $\lambda_{\min} = 3.0$, $\lambda_{\max} = 20.0$, $L_{\text{ref}} = 50.0$.

Design Decision: Why Adaptive Smoothing?

Fixed smoothing ($\lambda = \text{const}$) over-regularizes short texts and under-regularizes long texts. The adaptive formula (1) provides:

- **Short texts** ($L < 10$): $\lambda \approx 3.0$ — minimal smoothing preserves sparse signals
- **Medium texts** ($L \approx 50$): $\lambda \approx 20.0$ — full smoothing regularizes mass values
- **Long texts** ($L > 50$): λ saturates at 20.0 — prevents over-smoothing

The reference length $L_{\text{ref}} = 50$ corresponds to a typical Reddit post, the dominant training platform.

The effective length and log-dampened mass features are then:

$$L_{\text{eff}} = L + \lambda \quad (2)$$

$$B_{\text{pos_mass}} = \frac{\log(1 + M_{\text{pos}})}{\log(1 + L_{\text{eff}})} \quad (3)$$

$$B_{\text{neg_mass}} = \frac{\log(1 + M_{\text{neg}})}{\log(1 + L_{\text{eff}})} \quad (4)$$

$$B_{\text{density}} = \frac{\text{cnt}}{L_{\text{eff}}} \quad (5)$$

The logarithmic dampening in both numerator and denominator is crucial: it compresses the dynamic range of mass values, preventing a 200-token Reddit post from producing mass values $14\times$ larger than a 15-token tweet.

3.2.4 The Six Features of Channel B

Table 2: Channel B feature definitions.

Feature	Formula	Interpretation
B_pos_mass	$\log(1 + M_{\text{pos}}) / \log(1 + L_{\text{eff}})$	Stress-indicative pattern density
B_neg_mass	$\log(1 + M_{\text{neg}}) / \log(1 + L_{\text{eff}})$	Non-stress pattern density
B_polarity	$M_{\text{pos}} / (M_{\text{pos}} + M_{\text{neg}})$	Directional balance
B_density	$\text{cnt} / L_{\text{eff}}$	Pattern coverage ratio
B_load	$\log(1 + \text{total}) / \max(\log(1 + \text{cnt}), 1)$	Average pattern weight
B_abs	$\text{abs_count} / L_{\text{eff}}$	Absolutist term density

3.3 Channel C: Psycholinguistic Style (154 features)

Channel C is the richest channel, combining base psycholinguistic features with structural enhancements, open lexicons, and raw intensity signals.

3.3.1 Yule’s I: Length-Independent Lexical Diversity

Design Decision: Why Yule’s I Instead of TTR?

Type-Token Ratio (TTR = V/N) mathematically decays as text length increases: a 200-word text will almost always have lower TTR than a 15-word text, creating a severe structural bias when transferring from Reddit to Twitter. Yule’s I treats vocabulary distribution as a probabilistic urn model:

$$I = \frac{M_1^2}{M_2 - M_1} \quad (6)$$

where M_1 is the vocabulary size and $M_2 = \sum f_i^2$. This renders the diversity score asymptotically independent of document length, allowing TSS to consistently measure “rumination” and “lexical repetitiveness” across wildly different platform constraints.

3.3.2 Structural Negation Scope Balance

This feature operationalizes a clinical distinction from cognitive therapy:

$$\text{NegScopeBalance} = \frac{R(\text{Neg} \rightarrow \text{Verb}) - R(\text{Neg} \rightarrow \text{Adj})}{\text{TotalNegEvents} + \epsilon} \quad (7)$$

- **NEG→VERB** (“can’t do,” “don’t want”) signals **behavioral helplessness**—inability to act
- **NEG→ADJ** (“not happy,” “not okay”) signals **negative self-evaluation**—emotional assessment

The implementation scans `token_pos_pairs` for negation tokens followed by verbs (VB*) or adjectives (JJ*), using the expanded negation set from `config.py` (including informal forms: “cant,” “wont,” “aint”).

3.3.3 Sentence Rhythm Features

Stressed speech exhibits erratic rhythm—alternating between rapid short bursts and disjointed longer segments:

- `std_sentence_length`: Standard deviation of sentence word counts
- `short_burst_ratio`: Fraction of sentences with <5 words (fragmented thinking)
- `rhythm_coefficient`: Coefficient of variation (CV) of sentence lengths

3.3.4 Length-Robust Squashing Transform

$$C_{\text{out}} = \tanh(C_{\text{features}} \times 2.0) \quad (8)$$

Design Decision: Why tanh Squashing?

The multiplication by 2.0 before tanh pushes strong signals toward saturation (± 1), converting continuous features into soft binary triggers. This serves two purposes:

- **Cross-platform robustness:** A negation ratio of 0.15 on Reddit and 0.12 on Twitter both map to ~ 0.29 after squashing, reducing distributional shift
- **Outlier suppression:** Extreme values (e.g., a tweet that is 100% punctuation) are compressed to ± 1 rather than dominating the feature space

3.3.5 Open Lexicon Categories (112 features)

Channel C incorporates 56 open-source lexical categories (each producing count + ratio = 112 features), covering: emotions (anger, fear, sadness, joy, surprise, disgust), cognitive processes (certainty, uncertainty, insight, causation, discrepancy), social categories (family, friend, social), personal pronouns (I, we, you, they), temporal orientation (past, present, future), biological processes, and more. Multi-word phrases (e.g., “kind of,” “sort of”) are handled via dedicated matching before word-level tokenization.

4 The ML Pipeline and Algorithmic Decisions

tss/pipeline.py — 790 lines

4.1 L2 Normalization: From Quantity to Density**The Cross-Platform Length Problem**

Without normalization, a Reddit post with 50 stress-indicative POS bigrams produces feature values summing to ~ 50 . A tweet with 3 stress-indicative bigrams sums to ~ 3 . The classifier learns “high numbers = stress” from Reddit and then systematically under-predicts stress on Twitter. In early experiments, this caused Twitter recall to collapse to 0.099.

The fix is L2 normalization: $\mathbf{x}' = \mathbf{x} / \|\mathbf{x}\|_2$. After normalization, both vectors have unit norm ($\|\mathbf{x}'\|_2 = 1$), and the model compares *stress density* (the relative distribution of features) rather than *stress quantity* (the absolute sum).

4.2 Conditional Scaling: Protecting Channel B’s Sparsity**Design Decision: Why Disable StandardScaler for Channel B?**

After L2 normalization, all feature vectors have the same magnitude. StandardScaler then tries to make each feature’s variance equal to 1. For Channel B’s 6 sparse features, this multiplies small values by enormous scaling factors, effectively *re-introducing the length bias* that L2 normalization just removed.

The solution is conditional scaling:

- **Channels without B** (A, C, AC): Normalizer (L2) $\rightarrow$ StandardScaler
- **Channels with B** (B, BC, AB, ABC): Normalizer (L2) only

Listing 4: Conditional scaling implementation

```
1 if 'B' not in self.channel.upper():
2     self.scaler_ = StandardScaler(with_mean=False)
```

```

3     features = self.scaler_.fit_transform(features)
4     else:
5         self.scaler_ = None    # Disable scaler for Channel B

```

The `with_mean=False` parameter in `StandardScaler` preserves sparsity for Channel A's TF-IDF vectors.

4.3 ElasticNet and the Structural Sparsity Bias

Design Decision: Why ElasticNet for Channel B?

Channel B's 6-feature space requires careful regularization. Ridge (L2) would keep all 6 features alive, even if some carry noise. Lasso (L1) might zero out features too aggressively. ElasticNet provides the middle ground, with L1/L2 balance tuned via 5-fold internal cross-validation. The grid $[0.1, 0.5, 0.7, 0.9, 0.95]$ spans from nearly-Ridge (0.1) to nearly-Lasso (0.95). The consistent convergence to $l_1_ratio = 0.95$ across all Channel-B-containing models is a striking result: the algorithm autonomously discovers that the structural evidence is best represented by a sparse subset of features.

Listing 5: Channel-adaptive regularization

```

1 if uses_channel_b:
2     return LogisticRegressionCV(
3         cv=5, solver='saga', penalty='elasticnet',
4         l1_ratios=[0.1, 0.5, 0.7, 0.9, 0.95],
5         Cs=[self.C], class_weight='balanced',
6         max_iter=5000, scoring='f1', refit=True)
7 else:
8     return LogisticRegression(
9         C=self.C, penalty='l2', solver='lbfgs',
10        max_iter=2000, class_weight='balanced')

```

4.4 Dynamic Threshold Calibration

Rather than using the default 0.5 probability threshold, TSS searches $[0.20, 0.80]$ to find the threshold that maximizes F1 on out-of-fold (OOF) predictions. This is critical for two reasons:

1. **Class imbalance:** Reddit-combi's 12% stress prevalence means 0.5 is far from optimal
2. **LODO folds:** When holding out a domain, the class balance can shift substantially

The OOF calibration prevents leakage: thresholds are computed from predictions on data the model has never seen during that fold's training.

Listing 6: Threshold optimization (no leakage)

```

1 def optimize_threshold(y_true, y_proba,
2                        min_threshold=0.20, max_threshold=0.80):
3     best_thr, best_score = 0.5, 0.0
4     for thr in np.arange(min_threshold, max_threshold+0.005, 0.01):
5         y_pred = (y_proba >= thr).astype(int)
6         score = f1_score(y_true, y_pred, zero_division=0)
7         if score > best_score:
8             best_score = score
9             best_thr = thr
10    return best_thr, best_score

```

Domain-specific thresholds are computed and stored in artifacts for optional per-domain analysis, but are **not** used in headline cross-platform results—ensuring that reported Macro-F1 values reflect genuine cross-platform transfer without per-domain tuning.

5 Audit Protocols and Statistical Inference

scripts/03-06 — 7,086 lines total

5.1 Degree of Divergence (DoD): The DiD Adaptation

DoD adapts the Difference-in-Differences (DiD) framework from causal econometrics to quantify labeling-regime bias:

$$\text{DoD} = \Delta_{\text{human}} - \Delta_{\text{auto}} = \underbrace{(F1_{\text{human}}^{BC} - F1_{\text{human}}^A)}_{\text{structural advantage on human}} - \underbrace{(F1_{\text{auto}}^{BC} - F1_{\text{auto}}^A)}_{\text{structural advantage on auto}} \quad (9)$$

Design Decision: Why Instance-Level Bootstrap?

With only 4 datasets, aggregate-level tests have $N = 4$ —insufficient for parametric inference. Instance-level bootstrap ($N = 12,906$) resamples individual predictions ($y_{\text{true}}, y_{\text{pred}}$ pairs) to compute Macro-F1, then derives DoD from the resampled scores. This provides:

- $n = 10,000$ bootstrap iterations for stable confidence intervals
- Paired testing (same instances across channels)
- Benjamini–Hochberg FDR correction across all 27 hypothesis tests

Result: $\text{DoD}(BC-A) = 0.0374$, 95% CI $[0.0097, 0.0651]$, $p = 0.0032$, Cohen’s $d = 2.63$.
The bootstrap procedure operates as follows:

1. For each of $n = 10,000$ iterations, resample $N = 12,906$ instances with replacement
2. Compute Macro-F1 for each channel on each regime (human/auto) from the resampled predictions
3. Compute Δ_{human} and Δ_{auto} for the resampled data
4. Store $\text{DoD}_b = \Delta_{\text{human},b} - \Delta_{\text{auto},b}$
5. The 95% CI is the [2.5th, 97.5th] percentile of the 10,000 DoD values
6. The p -value is computed as $2 \cdot \min(\text{fraction} > 0, \text{fraction} \leq 0)$

Multiple comparison control: Across the full statistical suite, TSS conducts 27 hypothesis tests. Benjamini–Hochberg FDR correction is applied globally: the tests are ranked by raw p -value, and each threshold is adjusted as $\alpha_k = k \cdot \alpha / m$ where $m = 27$. Of 27 tests, 14 reject the null after FDR correction. The regime-permutation diagnostic ($p = 0.092$, FDR-adjusted $p = 0.166$) falls outside the rejection boundary, and is reported as a conservative supplementary audit rather than a primary claim-driving result.

5.2 The Lexical Interference Paradox: Statistical Evidence

The paradox is established through paired instance-level bootstrap on the $C \rightarrow AC$ transition:

Table 3: Lexical Interference test results.

Regime	Mean drop (C→AC)	95% CI	p (two-sided)
Human-labeled	0.0720	[0.052, 0.092]	<1e-4
Auto-labeled	-0.0201	[-0.033, -0.007]	0.0024

The two-sided p -values are computed as $2 \cdot \min(p_{\text{sign}}, 1 - p_{\text{sign}})$ to align CI interpretation with hypothesis testing under the same bootstrap draws.

Understanding the Large Cohen’s d_z Values

TSS reports paired-samples Cohen’s $d_z = \mu_D / \sigma_D$, where D is the vector of per-instance prediction differences. Given the large evaluation set ($N = 12,906$) and the paired nature, the standard deviation of differences (σ_D) is exceptionally small. This mathematically yields large standardized effect sizes (e.g., $d = 2.63$ for DoD). These should be interpreted as evidence of **highly consistent discriminability across instances** rather than absolute clinical magnitude.

5.3 Interventional Masking: Quasi-Causal Evidence

The masking suite (`03_masking_suite.py`) implements five text transformations that *intervene* on the text rather than merely ablating features:

Listing 7: Masking implementation (`03_masking_suite.py`)

```

1 def apply_mask(token_pos_pairs, mode, rng=None):
2     if mode == 'none':
3         return token_pos_pairs
4     elif mode == 'pos_only':
5         # Replace every word with its POS tag -> destroys ALL lexical
6         # content
7         return [(pos, pos) for token, pos in token_pos_pairs]
8     elif mode == 'function_only':
9         # Keep ONLY function words (pronouns, prepositions, determiners)
10        return [(t, p) for t, p in token_pos_pairs
11                if p in FUNCTION_POS_PENN]
12    elif mode == 'content_only':
13        # Keep ONLY content words (nouns, verbs, adjectives)
14        return [(t, p) for t, p in token_pos_pairs
15                if p in CONTENT_POS_PENN]
16    elif mode == 'random_pos':
17        # Shuffle POS tags randomly -> destroys syntactic structure
18        tokens = [t for t, _ in token_pos_pairs]
19        pos_tags = [p for _, p in token_pos_pairs]
20        rng.shuffle(pos_tags)
21        return list(zip(tokens, pos_tags))

```

Table 4: Masking modes and their causal interpretation.

Mode	Transformation	Causal Question
none	Original text	Baseline
pos_only	Replace words with POS tags	Does performance survive destruction of all lexical content?
content_only	Keep only nouns, verbs, adjectives	Does topical content alone suffice?
function_only	Keep only function words	Do unconscious function-word dynamics carry the signal?
random_pos	Shuffle POS tags randomly	Negative control: does arbitrary syntax retain signal?

Key result: C/pos_only retains 99.3% of Channel C’s performance on Dreddit-test (0.734 vs 0.739), proving that structural signals operate independently of vocabulary.

The random_pos Anomaly

random_pos occasionally outperforms the unmasked baseline by a marginal fraction (e.g., 0.742 vs 0.739 on Dreddit-test). TSS interprets this as an extreme form of **structural dropout**: by scrambling rigid, dataset-specific syntactic patterns, the model is forced to unlearn overfitted morpho-syntactic noise, inadvertently yielding a slight generalization gain. This does *not* imply that random syntax is informative—it implies that the specific POS ordering in Dreddit contains some exploitable regularity that scrambling disrupts.

5.4 The Permutation Tension

The regime-permutation diagnostic ($p = 0.092$, FDR-adjusted $p = 0.166$) tests a different null than bootstrap. Bootstrap evaluates whether the DoD effect is stable across instance resampling; permutation tests whether the human/auto regime *ordering* could arise under the channel-design constraints. The marginal permutation result is expected given the small number of regime groups (2) and the conservative test design. TSS prioritizes bootstrap CIs for effect estimation while reporting the permutation result as a conservative supplementary audit.

5.5 Unsupervised Phenotyping and Leakage Auditing

A key robustness signature is **cross-platform structural invariance**: BC achieves virtually identical Macro-F1 on the two human-regime datasets (0.690 on Dreddit-test vs 0.690 on Twitter-gold; $\Delta \approx 0.0002$), suggesting minimal degradation under platform shift when relying on structure and style rather than lexical topics. This invariance is a direct consequence of the engineering decisions in Channels B and C: L2 normalization, adaptive smoothing, Yule’s I, and tanh squashing all contribute to platform-robust feature representations.

K-Means ($K = 3$) on Channel C’s 154-dimensional style space yields:

- **ARI = 0.980** (95% CI [0.961, 0.991]): Highly stable clusters under bootstrap
- **Cluster–dataset AMI = 0.281** ($p = 0.0005$): Non-trivial platform entanglement
- **Cluster–label AMI = 0.084** ($p = 0.0005$): Weak but significant stress signal

The verdict is “artifact”—the clusters are stable but partly platform-shaped. TSS treats this as an explicit **leakage audit** rather than a clinical discovery.

5.6 Channel Orthogonality

Instance-level Spearman ρ across all channel pairs yields mean $|\rho| = 0.595$, confirming partial (not full) independence. The weakest primary pair (A vs B, $\rho = 0.296$) validates the intended factorization: morpho-syntax captures a partially distinct signal from lexical surface-form.

5.7 LLM Baseline Instability: Regime-Sensitive Collapse

LLaMA-3-8B (3-shot) provides a critical control condition. While it achieves strong headline Macro-F1 in some regimes (0.800 on Twitter-gold), its precision–recall profile is catastrophically unstable across platforms:

Table 5: LLaMA-3 regime-sensitive instability.

Dataset	F1	Precision	Recall	Interpretation
Dreaddit (H)	0.613	0.606	0.984	Over-sensitivity: predicts nearly everything as stressed
Twitter-auto (A)	0.522	0.846	0.203	Under-sensitivity: misses 80% of stressed cases
Twitter-gold (H)	0.800	0.780	0.680	Moderate balance

Why LLM Instability Matters for the Audit Argument

LLaMA’s recall swing from 0.984 (Dreaddit) to 0.203 (Twitter-auto) demonstrates that even billion-parameter models are subject to regime-sensitive instability. The model’s internal decision boundary shifts dramatically depending on which platform and labeling regime it encounters. This is precisely the failure mode that TSS’s DoD metric is designed to detect and quantify. TSS does not claim to *outperform* LLMs—it claims to *diagnose why they fail differently across regimes*.

MentalBERT shows a parallel pattern: strong on Dreaddit (0.801) but collapsing on Twitter-auto (0.418). The paired McNemar tests confirm these are statistically significant differences, not sampling artifacts. Critically, TSS-C *outperforms* MentalBERT on Twitter-auto (+0.086 Macro-F1, $p < 10^{-12}$), demonstrating that structural features provide genuine cross-platform resilience where transformer models relying on lexical patterns fail.

5.8 LODO Cross-Stressor Evaluation: Detailed Protocol

The Leave-One-Domain-Out (LODO) protocol on Dreaddit’s 5 stressor domains (abuse, anxiety, financial, PTSD, social) implements nested validation:

1. **Train** on 4 domains (60% of combined data)
2. **Calibrate threshold** on held-in validation split (20%)
3. **Evaluate in-domain** on held-in test split (20%)
4. **Evaluate out-of-domain** on the entire 5th held-out domain

Channel C achieves the most stable out-of-domain performance (mean Macro-F1 = 0.685, std = 0.015 across 5 folds), supporting the cross-stressor cognitive invariance hypothesis (H2). By contrast, Channel A (lexical) shows higher variability (std = 0.042), reflecting topical sensitivity: when the abuse domain is held out, the lexical transfer gap reaches 0.063 in Macro-F1.

6 Qualitative Conflict Zone Analysis

6.1 Methodology: Systematic Shift-Case Extraction

Rather than cherry-picking anecdotal examples, TSS implements a systematic extraction pipeline (`02.1_extract_error_cases.py`) that identifies **682 shift cases** where channels disagree. The extraction logic operates on instance-level predictions:

Listing 8: Shift-case extraction logic

```

1 # For each instance in evaluation set:
2 for i in range(len(y_true)):
3     a_correct = (y_pred_A[i] == y_true[i])
4     bc_correct = (y_pred_BC[i] == y_true[i])
5     if bc_correct and not a_correct:
6         shift_cases_bc_wins.append(i) # Structure wins
7     elif a_correct and not bc_correct:
8         shift_cases_a_wins.append(i) # Lexicon wins

```

These shift cases are exported to `error_analysis_qualitative.xlsx` with complete audit trails: instance IDs, per-channel predictions, probabilities, true labels, and the raw text (anonymized). The workbook contains separate sheets for each conflict type, enabling systematic rather than anecdotal error taxonomy.

6.2 Three Failure Archetypes

The 682 shift cases cluster into three recurring archetypes, each revealing a distinct failure mode of lexical-first classification:

6.2.1 Archetype 1: Semantic Hallucination (Keyword Spotting)

“call us telephone receiver at +971 4 324 3244 to consult a medical professional. EyeHealth MentalHealth Ophthalmologist Stress DryEye EyeTwitch”

Truth: 0 (not stressed) | Channel A: 1 (false positive) | Channel B/C: 0 (correct)

Channel A’s character n-grams detect dense concentrations of stress-related hashtags (#MentalHealth, #Stress) and triggers a false positive. Channel B/C correctly rejects this text by detecting: imperative syntax (“call us”), zero first-person pronouns, no cognitive fragmentation markers, and a stable declarative sentence rhythm. This exemplifies why SSI (Semantic–Structural Intersection) breaks down outside support communities.

6.2.2 Archetype 2: Implicit Stress (Missing Vocabulary)

“I am on the edge of committing and I have a few hours remaining to decide whether I end my life.”

Truth: 1 (stressed) | Channel A: 0 (false negative) | Channel B/C: 1 (correct)

This text contains *no* mental health keywords (“suicide,” “depression,” “anxiety”) yet expresses severe distress. Channel C flags it via: dense first-person pronouns (I×3, my×1), absolutist framing (“edge,” “end”), temporal urgency (“a few hours remaining”), and specific POS action sequences (PRON→VB patterns). This is precisely the “silent stress” that lexical systems systematically miss.

6.2.3 Archetype 3: Boundary Collapse (Context Contagion)

“I man the front desk and my title is HR Customer Service Representative... The rest is dealing with people who walk in and need help with benefits, retirement, etc.”

Truth: 0 (not stressed) | Channel A: 1 (false positive) | Channel B/C: 0 (correct)

Channel A triggers on “need help” and “dealing with people”—phrases that co-occur with stress in training data but here describe an objective professional context. Channel B/C correctly identifies this as a stable declarative narrative with professional tone, zero structural anxiety markers, and no first-person emotional engagement.

7 Efficiency, Privacy, and Reproducibility

7.1 Green AI: Training Time Analysis

Table 6: Efficiency profile by channel.

Channel	Features	Training (s)	Regularization	l_1_ratio
A	500	17.6	L2 (Ridge)	—
B	6	1007.9	ElasticNetCV	0.95
C	154	27.5	L2 (Ridge)	—
BC	160	335.9	ElasticNetCV	0.95

Why Does Channel B Take 1007 Seconds With Only 6 Features?

Channel B’s training time is dominated by the ElasticNetCV hyperparameter search: 5-fold internal CV $\times$ 5 l_1_ratio values $\times$ 5 outer CV folds = 125 model fits, each using the SAGA solver (required for ElasticNet) with `max_iter=5000`. Once fitted, inference over 6 features is near-instantaneous (<1ms per sample). This is a one-time cost that enables data-driven regularization selection rather than arbitrary hyperparameter choices.

7.2 Privacy-by-Design

Channel B provides a complete content-free screening pipeline:

1. Text is parsed by spaCy to extract POS tags and dependency structure
2. Only POS bigrams and abstract SVO triples are retained
3. All lexical tokens are discarded before model input
4. Sensitive patient text never leaves the clinical perimeter

Combined with the `pos_only` masking result (99.3% performance retention), this yields a practical blueprint for privacy-preserving CMH auditing.

7.3 Reproducibility Checklist

Table 7: Reproducibility parameters.

Parameter	Value
Random seed (global)	42
Bootstrap iterations (DoD)	10,000
Bootstrap iterations (CI)	1,000–2,000
CV folds (outer)	5 (StratifiedKFold)
CV folds (ElasticNet internal)	5
Threshold search range	[0.20, 0.80] step 0.01
spaCy model	en_core_web_sm
Python	3.10+
Key dependencies	scikit-learn, scipy, pandas, spaCy, VADER

7.4 Pipeline Execution

The entire TSS pipeline is automated via `run_all.sh`:

Listing 9: End-to-end pipeline execution

```

1 ./run_all.sh                # Full pipeline
2 ./run_all.sh --skip-prep    # Skip data preparation
3 ./run_all.sh --channels BC  # Train only BC channel

```

Stages: (1) Environment check → (2) Unit tests → (3) Data preparation → (4) Training + evaluation → (5) Masking suite → (6) Statistical analysis → (7) Advanced analysis.

8 Dataset Registry and Auto-Discovery

`tss/dataset_registry.py` — 194 lines

The Dataset Registry eliminates dataset-handling errors through a single source of truth:

- **Canonical mappings** for 5 core corpora (`dreaddit_train/test`, `twitter`, `twitter_gold`, `reddit_combi`)
- **Auto-discovery** of new CSVs dropped into `data/`: the system infers `label_type` (human/auto) and `platform` (reddit/twitter) from filename heuristics
- **10 dataset aliases** resolved at runtime, logged for auditability

Heuristic rules: filenames containing “gold/human/expert/dreaddit” → human; “combi/auto/distant/proxy” → auto. This ensures that adding a new dataset (e.g., `facebook_gold.csv`) requires zero code changes.

9 Codebase Architecture Map

9.1 Module Dependency Graph

The TSS codebase comprises 11,651 lines across 11 Python modules with a clean layered architecture:

Table 8: Complete codebase inventory.

Layer	Module	Lines	Responsibility
Core	tss/config.py	210	Constants, NRC-VAD
	tss/features.py	1,184	Channel A/B/C
	tss/pipeline.py	790	Classifier, DoD
	tss/dataset_registry.py	194	Auto-discovery
	tss/open_lexicons.py	209	56 lexical categories
Scripts	01_prepare_data.py	1,306	Data cleaning
	02_train_evaluate.py	1,558	Training, LODO
	03_masking_suite.py	1,670	Masking experiments
	04_statistical_analysis.py	1,632	DoD, bootstrap
	05_advanced_analysis.py	2,412	Clustering, SHAP
	06_baseline_comparison.py	486	LLM baselines
	run_all.sh	—	Orchestration

9.2 Data Flow

1. **Raw** → **Processed** (01): CSVs are cleaned, deduplicated, POS-tagged, and feature-engineered into `data/processed/`
2. **Processed** → **Models** (02): 5-fold CV trains 7 channel models, exports to `artifacts/` as joblib files
3. **Models** → **Predictions** (03): All channels $\times$ all masks $\times$ all datasets evaluated, instance-level predictions saved
4. **Predictions** → **Statistics** (04): Instance-level bootstrap, DoD, McNemar, FDR correction
5. **Statistics** → **Analysis** (05): Clustering, SHAP, phenotyping, divergence visualization
6. **Predictions** → **Baselines** (06): Paired comparison against MentalBERT and LLaMA

9.3 Key Design Patterns

- **Scikit-learn Protocol:** All channels implement `BaseEstimator/TransformerMixin` with `fit/transform/get_feature_names_out`, enabling seamless integration with sklearn pipelines
- **Immutable Configuration:** All word sets in `config.py` use `FrozenSet` to prevent accidental mutation at runtime
- **Lazy Loading:** spaCy models for SVO extraction are loaded once and cached as class-level singletons (`_spacy_nlp`)
- **Fail-Safe Parsing:** `parse_token_pos_pairs()` aggregates malformed-string warnings rather than crashing, enabling graceful degradation on noisy CSV data
- **JSON Serialization:** A custom `json_default()` handler converts NumPy types to Python natives, preventing serialization failures in result export

10 Conclusion: Engineering Philosophy

TSS embodies a specific engineering philosophy: **every design decision serves the epistemic audit mission**. L2 normalization exists because cross-platform transfer is the test. Conditional scaling exists because Channel B’s sparsity is the evidence. ElasticNet converges to $l_1_ratio = 0.95$ because structural

traces are inherently sparse. The tanh squashing exists because cross-platform distributional shift is the threat. The Dataset Registry auto-discovers corpora because leakage is the enemy.

The system’s 11,651 lines of code produce a single number—DoD = 0.0374 ($p = 0.0032$)—that no black-box model can replicate: a statistically grounded, interpretable, regime-aware measurement of how much your labels diverge from clinical reality.

10.1 Summary of Key Engineering Decisions

Table 9: Summary of key engineering decisions.

Decision	Implementation	Rationale
No undersampling	<code>class_weight='balanced'</code>	Preserve epidemiological reality
Char n-grams	<code>char_wb, (3,5), k=500</code>	Sub-word robustness
POS bigrams only	<code>PRP_VBP not feel_VB</code>	Privacy + domain agnosticism
Adaptive smoothing	$\lambda \in [3, 20]$	Length-invariant Channel B
Log-dampened mass	$\frac{\log(1+M)}{\log(1+L_{\text{eff}})}$	Compress dynamic range
Yule’s I over TTR	$I = M_1^2 / (M_2 - M_1)$	Length-independent diversity
NegScope balance	NEG→VB vs NEG→JJ	Helplessness vs self-eval
Tanh squashing	$\tanh(C \times 2)$	Cross-platform outlier suppression
L2 normalization	<code>Normalizer('l2')</code>	Quantity → density
Conditional scaler	Disabled for Channel B	Protect sparsity
ElasticNet for B	$l_1\text{ratio} \rightarrow 0.95$	Data-driven sparsity
Threshold [0.2,0.8]	OOB calibration	Handle extreme imbalance
Instance bootstrap	$n=10K, N=12,906$	Robust DoD inference
Decontamination	Exact-match text	Zero leakage
Auto-discovery	Filename heuristics	Zero-code dataset addition

“Structure reveals stress. The rest is commentary.”